

OOPS IN PROJECT

Bus Booking System — Full Stack Application

Angular .NET Web API · PostgreSQL

Assignment Task

Pick the project that you did and explain what went well and what did not from the prospect of partial class, abstract class, Model preparation and more. Give examples with code and your thoughts.

1. Project Overview

Bus Booking System is an end-to-end web-based application that mimics actual bus travels booking processes in the real world. This application has been developed using **Angular (frontend)**, **.NET 10 Web API (backend)**, and **PostgreSQL (database)**.

This document analyzes the implementation of Object-Oriented Programming (OOP) concepts in the Bus Booking System. It evaluates which concepts were applied, where they appear in the codebase, and whether their usage aligns with real-world business logic requirements.

There are three types of users supported by the application – **Customers, Operators, and Admin**.

1.1 Folder Structure

The backend is cleanly organised into dedicated folders, each with a single responsibility:

Folder	Purpose
Backend/Models/	C# entity classes that map to PostgreSQL tables (Booking, Bus, User, Feedback, ...)
Backend/Controllers/	ASP.NET controllers that expose REST API endpoints
Backend/Services/	Business-logic classes (EmailService) — separated from controllers
Backend/Interfaces/	C# interfaces that define service contracts (IEmailService)
Backend/Data/	Entity Framework AppDbContext — database session / DbSet definitions
Frontend/src/app/pages/	Angular page components (Home, Login, Seat, Payment, Ticket, ...)
Frontend/src/app/components/	Shared UI components (Navbar)
Frontend/src/app/services/	Angular services that call the backend API

2. Interface Abstraction

Status: Used and correctly implemented

3.1 Definition

An interface is like a **contract** that tells what a class *must do*, but not how it does it.

3.2 Where is it used? — Backend/Models/Booking.cs

Every model class in the Models/ folder applies encapsulation. Booking.cs is the clearest example — it groups every piece of data that belongs to a seat booking into one self-contained class.

```
// File: Backend/Models/Booking.cs
public class BookingController(AppDbContext context, IEmailService
emailService) : ControllerBase
{
    [HttpPost("pay-group/{groupId}")]
    public async Task<IActionResult> PayGroup(int groupId)
    {
        // ... mark seats as Paid ...
        emailService.SendBookingEmail(user.Email, "Booking Confirmed",
emailBody);
        return Ok();
    }
}
```

3.3 How it helps

The controller depends on the **interface**, not the concrete implementation.
It knows **what to do (send email)**, but not **how it is done (SMTP/Gmail)**.

3.4 What I understand better now

In my opinion, this decouples business logic from implementation and enables testing with mock services also improves scalability.

4. Abstract Class

Status: Not Used (Justified)

Each controller has a distinct responsibility:

- Booking → seat lifecycle
- Admin → platform control
- Operator → business operations

Benefits:

- Centralized authorization logic
- Eliminates duplication
- Improves security

5. Partial Class

Status: Not Used (Justified)

- Classes are small and focused
- No auto-generated code
- Already well-structured via folders

Partial classes are **not required** here — using them would not improve my business logic.

6. Generics

Status: Used (Framework Level)

6.2 Where used?

Generics are used through:

- Entity Framework (DbSet<T>)
- LINQ (Where<T>, ToList<T>)
- Dependency Injection

```
public DbSet<Booking> Bookings { get; set; }  
public List<Booking> Bookings { get; set; }  
builder.Services.AddScoped<IEmailService, EmailService>();
```

Benefits

- Standardizes API responses
- Simplifies frontend handling
- Improves maintainability

7. Indexer

Status: Used Implicitly (Not Custom Defined)

7.3 Where used?

```
// indexer on IConfiguration  
_fromEmail = config["EmailSettings:FromEmail"];
```

Not required currently because:

- Seat logic is handled in frontend
- Database handles indexing

8. Model Preparation

Status: Partially present.

8.1 Where used?

```
// where used and went most well
[Table("bookings")]
public class Booking
{
    [Column("id")] public int Id { get; set; }
    [Column("status")] public string Status { get; set; }
    [Column("lock_expiry")] public DateTime? LockExpiry { get; set; }
}
```

Why This Matters

- Encapsulation improves
- Reduces duplication
- Keeps logic consistent

10. Conclusion

This project shows a grasp of database modeling and object-oriented programming at the framework level. It does not make full use of key object-oriented design principles like encapsulation and behavior-driven models.

Making the models include domain logic would greatly improve the system's design. It would make the system more scalable easier to maintain and more in line, with world object-oriented programming practices.

11. Summary — All OOP Concepts at a Glance

OOP Concept	File / Location	Status	Key points
Interface	EmailService	Already Present	Correct usage
Abstract Class	-	Not present	Missed opportunity
Partial Class	-	Not present	Not needed
Generics	EF, LINQ	Already Present	Used via framework
Indexer	IConfiguration	Partial Usage	Implicit only
Model Prep	All models	Scope for improvement	Needs improvement